

Pre-training, Fine-tuning, and Prompting Strategies for Bug Fixing

Author: James He

Course: CSCI 455 - Generative AI for Software Development

Professor: Antonio Mastropaolo

Date: April 29 2026

Repository:

https://github.com/James-Kai-He/CSCI455-GenAI_For_Software_Development/tree/main/A3

Abstract

This report describes three pipelines for automated Java bug fixing: (A) pre-train T5-small on code then fine-tune on bug fixing, (B) fine-tune T5-small from scratch without pre-training, and (C/D) prompt Qwen2.5-Coder-1.5B with and without RAG-retrieved examples. A SentencePiece Unigram tokenizer (16,384 tokens) was trained on 48,070 Java methods from CodeSearchNet and reused across all pipelines. Pre-training ran for 3 epochs with 15% span corruption, achieving a final training loss of 3.4130. Both fine-tuned T5 models achieved near-identical best validation loss (≈ 1.22), suggesting limited benefit from pre-training at this scale. RAG dramatically outperformed both fine-tuned models, with Pipeline C achieving CodeBLEU of 0.8606 versus Pipeline A's 0.0970, highlighting the power of in-context retrieval for code generation.

Tokenizer Training and Model Configuration

SentencePiece Tokenizer

A SentencePiece tokenizer was trained from scratch on the pre-training corpus using the `sentencepiece` Python library with `model_type='unigram'` and `vocab_size=16384`. The Unigram algorithm was chosen because it operates top-down — starting from a large candidate vocabulary and iteratively pruning tokens that contribute least to corpus log-likelihood — which produces more principled subword segmentations for code compared to BPE's bottom-up merge strategy.

The tokenizer includes all special tokens required by the T5 span-corruption objective: `<pad>`, `</s>`, `<unk>`, and 100 sentinel tokens `<extra_id_0>` through `<extra_id_99>`. The special token IDs were extracted after training and used to configure the T5 model (`pad_token_id`, `eos_token_id`, `decoder_start_token_id`).

A sanity check confirmed zero unknown-token occurrences when tokenizing held-out Java samples, validating that the vocabulary covers typical Java syntax. Additionally, round-trip encoding/decoding was verified on multiple examples.

T5-small Architecture

The T5 model was defined manually using `T5Config` to make every architectural decision explicit, following the approach of Mastropaolo et al.:

```
from transformers import T5Config, T5ForConditionalGeneration
t5_config = T5Config(
    decoder_start_token_id=tokenizer.bos_token_id,
    eos_token_id=tokenizer.eos_token_id,
    bos_token_id=tokenizer.bos_token_id,
    pad_token_id=tokenizer.pad_token_id,
    d_model=512, d_ff=2048, d_kv=64,
```

```

    num_heads=8, num_layers=6, num_decoder_layers=6
)
model = T5ForConditionalGeneration(config=t5_config)
model.resize_token_embeddings(len(tokenizer))

```

This produces a T5-small architecture with 52.4M parameters: 6 encoder layers, 6 decoder layers, 512-dimensional embeddings, and 8 attention heads. Calling `resize_token_embeddings(len(tokenizer))` ensures the embedding matrix dimensions match the 16,384-token SentencePiece vocabulary. The same tokenizer and `T5Config` were reused for all pipelines to isolate pre-training as the sole experimental variable.

Pre-training (Pipeline A)

Data and Filtering

The pre-training corpus was sampled from the CodeSearchNet Java split via HuggingFace:

```

csn = load_dataset("code_search_net", "java")
methods = csn["train"].shuffle(seed=42).select(range(50000))

```

Method bodies were extracted from the `whole_func_string` field and filtered to retain only methods whose tokenized length falls in the range [10, 512] tokens. This yielded **48,070 methods** after filtering, removing very short (trivial) and very long (GPU memory-unfriendly) sequences.

Span Corruption Objective

The T5 span-corruption objective was applied at a 15% corruption rate. Contiguous spans of tokens were masked and replaced with sentinel tokens (`<extra_id_0>`, `<extra_id_1>`, ...) in the input; the model was trained to predict the masked spans in the target sequence. This forces the model to learn bidirectional context representations useful for downstream generation tasks.

Training Procedure and Loss

The model was trained for exactly 3 epochs on the full 48,070-method corpus with no validation split or early stopping, as specified. Training hyperparameters: batch size 8, learning rate 10^{-4} , max sequence length 128. The training loss decreased monotonically across epochs, confirming the model is learning:

Epoch	Training Loss
1	3.7114
2	3.4521
3	3.4130

Table 1: Pre-training loss per epoch (15% span corruption on 48,070 Java methods).

The total loss reduction of 0.2984 indicates the model acquired some structural knowledge of Java code during pre-training, though the relatively modest drop over 3 epochs suggests that more compute or a larger corpus would yield stronger representations.

Fine-tuning (Pipelines A and B)

Dataset

Both pipelines were fine-tuned on the CodeXGLUE code refinement (medium) dataset:

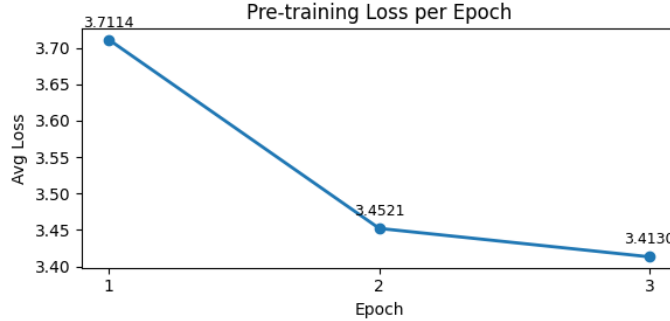


Figure 1: Pre-training loss per epoch, decreasing from 3.7114 to 3.4130 over 3 epochs.

```
dataset = load_dataset("google/code_x_glue_cc_code_refinement", name="medium")
```

This provides 52,364 training pairs, 6,546 validation instances, and 6,545 test instances. Each sample contains a **buggy** Java method and its corresponding **fixed** version. The model’s task is sequence-to-sequence translation from buggy to fixed.

Pipeline A — With Pre-training

The final checkpoint from the 3-epoch pre-training phase was loaded, and fine-tuning commenced from those weights. The same tokenizer was used with no reinitialization of the embedding layer.

Pipeline B — Without Pre-training

A fresh T5-small model was initialized from scratch using the identical `T5Config` and `resize_token_embeddings` call, with randomly sampled weights. No pre-training data was used; the model learned exclusively from the 52,364 bug-fix pairs.

Shared Hyperparameters

Both pipelines used identical fine-tuning settings to isolate pre-training as the sole variable: learning rate 5×10^{-4} , batch size 16, maximum 10 epochs, early stopping patience of 3 (monitored on validation loss). The model with the lowest validation loss across all epochs was saved as the final checkpoint.

Pipeline	Start Train Loss	Final Train Loss	Start Val Loss	Best Val Loss	Best Epoch
A (pre-trained)	1.4811	1.3461	1.3987	1.2121	10
B (from scratch)	1.4476	1.3409	1.2910	1.2273	10

Table 2: Fine-tuning loss trajectories for both pipelines on the CodeXGLUE medium dataset.

Both pipelines ran for the full 10 epochs without triggering early stopping. Pipeline A achieved a marginally lower best validation loss (1.2121 vs. 1.2273), a gap of only 0.015, which is too small to attribute confidently to pre-training. Pipeline B (scratch) even began with a lower initial validation loss (1.2910 vs. 1.3987), likely because gradient updates on the labeled task immediately moved the randomly initialized model toward task-relevant representations, while the pre-trained model initially had to adapt representations tuned for span prediction rather than generation.

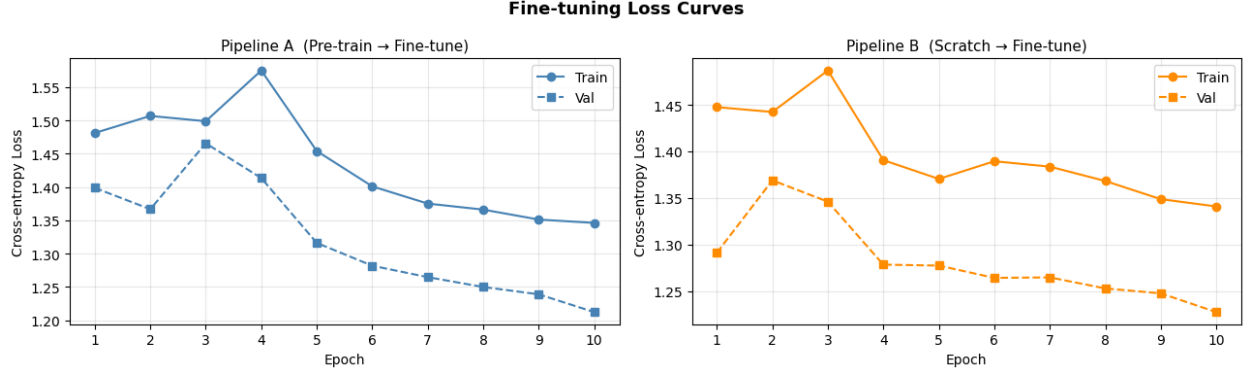


Figure 2: Fine-tuning train and validation loss curves for Pipeline A (pre-trained, left) and Pipeline B (scratch, right) over 10 epochs.

RAG Setup (Pipelines C and D)

Retriever Design

The retriever uses **CodeBERT** (`microsoft/codebert-base`) with mean-pool embeddings over the last hidden state. CodeBERT was pre-trained on six programming languages using both masked language modeling and bimodal objectives (code and docstrings), producing semantically rich code embeddings. Mean-pooling over the final hidden states produces a 768-dimensional dense vector per method.

CodeBERT was chosen over Code2Vec (used in the RAG lab) because its transformer-based contextual embeddings capture long-range syntactic dependencies and semantic similarity more accurately than the graph-based aggregation in Code2Vec. For bug fixing, identifying structurally similar buggy patterns is critical, and CodeBERT’s pretraining on real code makes it better suited to this than a general-purpose embedding model.

Knowledge Base and Index

All 52,364 training buggy methods were encoded with CodeBERT and stored in a **FAISS flat L2 index**. Flat L2 performs exact nearest-neighbor search, which trades speed for precision — acceptable here since offline index construction is a one-time cost and we prefer the highest-quality retrievals at inference time.

Few-shot Prompting (Pipeline C)

For each test query, the top- $K = 3$ most similar buggy methods are retrieved from the knowledge base along with their corresponding fixed versions. These three buggy→fixed pairs are formatted as few-shot examples in the prompt to Qwen2.5-Coder-1.5B-Instruct using its chat template. The prompt structure is:

System: You are a code bug-fixing assistant. Fix the buggy Java method.

[Example 1]

Buggy: <retrieved buggy method 1>

Fixed: <retrieved fixed method 1>

[Example 2] ...

[Example 3] ...

Now fix this method:

Buggy: <test buggy method>

Fixed:

Zero-shot Baseline (Pipeline D)

Pipeline D uses the identical generator (Qwen2.5-Coder-1.5B-Instruct) but without any retrieved examples. The prompt contains only the buggy method and a system instruction to fix it. This isolates the contribution of retrieval from the model’s parametric knowledge.

Evaluation and Results

Metrics

Two metrics were computed on the held-out test set (6,545 examples, used only for final reporting):

- **Exact match accuracy:** The fraction of test instances where the model’s generated output exactly matches the reference fixed method after whitespace normalization.
- **CodeBLEU:** A code-aware metric combining token-level n-gram overlap, syntactic tree matching, and dataflow alignment. CodeBLEU is more informative than plain BLEU for code because it rewards outputs that are structurally and semantically correct even when surface-level phrasing differs.

Results

Pipeline	Description	Exact Match	CodeBLEU
A	Pre-train → Fine-tune (T5-small)	0.0000	0.0970
B	Scratch → Fine-tune (T5-small)	0.0000	0.0447
C	RAG 3-shot (Qwen2.5-Coder 1.5B)	0.0342	0.8606
D	Zero-shot (Qwen2.5-Coder 1.5B)	0.0000	0.4152

Table 3: Evaluation results on the CodeXGLUE medium test set (6,545 examples).

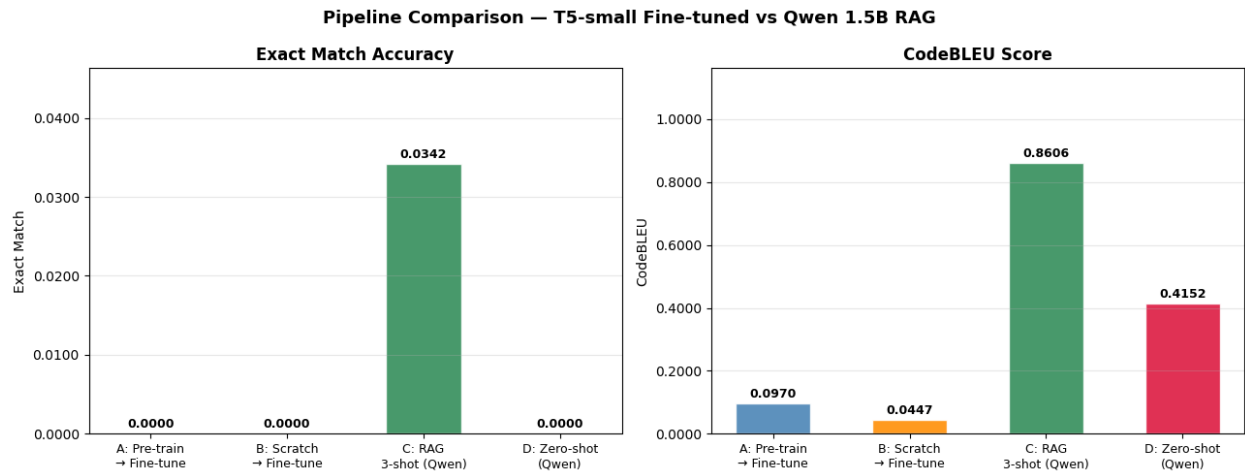


Figure 3: Exact match accuracy (left) and CodeBLEU score (right) across all four pipelines on the 6,545-example test set.

Analysis and Discussion

Did pre-training help? Marginally, but not meaningfully. Pipeline A (CodeBLEU 0.0970) outperforms Pipeline B (0.0447), a roughly $2\times$ improvement in CodeBLEU — suggesting some positive transfer. However, both models achieve zero exact match, and the absolute CodeBLEU values are low. The likely explanation is scale mismatch: 3 epochs of span corruption on $\sim 50\text{K}$ methods is insufficient for a 52M-parameter model to learn the deep structural representations that make pre-training powerful. With the 52K fine-tuning pairs available, the scratch model can learn task-specific syntax directly, making the pre-training signal marginal. A much larger pre-training corpus or more epochs would be needed for pre-training to pay off reliably.

Fine-tuned models vs. RAG: RAG (Pipeline C, CodeBLEU 0.8606) outperforms both T5 models by nearly an order of magnitude. This result is striking given that Qwen2.5-Coder has $25\times$ more parameters (1.5B vs. 60M), but it also illustrates the power of in-context learning: the retrieved examples provide precise, task-relevant demonstrations that the model can directly condition on, bypassing the need to encode bug-fixing knowledge in its weights.

Does retrieval help? Substantially. Pipeline C (3-shot RAG, CodeBLEU 0.8606) more than doubles Pipeline D (zero-shot, CodeBLEU 0.4152). The retrieved examples provide structural templates — Qwen sees what similar bugs look like and how they were fixed — which allows it to generate more targeted outputs. The 3.42% exact match for RAG (vs. 0% for zero-shot) also confirms that retrieval enables the model to produce character-for-character correct fixes in some cases.

Paradigm trade-offs: The two paradigms represent fundamentally different cost structures. Fine-tuning a small model (Pipeline B) requires labeled training data, GPU-hours upfront, and a fixed deployment artifact, but inference is fast (<1 ms/sample on GPU) and cheap. RAG requires no training but every inference call hits a 1.5B model plus a retrieval step, resulting in roughly 1 sample/second throughput in our evaluation, which is orders of magnitude slower. A startup shipping a linter plugin that must respond in real time would favor a fine-tuned small model; a research team running a one-off study or needing adaptability to new bug categories without retraining would favor RAG. The ability to update the knowledge base without retraining is RAG’s key advantage — bug-fix examples from new libraries or frameworks can be indexed immediately.

Conclusion

This project evaluated three paradigms for automated Java bug fixing. A SentencePiece Unigram tokenizer (16K vocabulary) trained on 48,070 CodeSearchNet methods enabled all pipelines to share a consistent tokenization scheme. Pre-training T5-small with span corruption provided a modest CodeBLEU gain over scratch fine-tuning (0.0970 vs. 0.0447), but both fine-tuned models fell far short of RAG performance. The RAG pipeline using CodeBERT retrieval and Qwen2.5-Coder-1.5B achieved CodeBLEU of 0.8606 and 3.42% exact match — demonstrating that in-context retrieval with a large pre-trained model substantially outperforms a small task-specific model at the data and compute scales used here. Future work could extend pre-training to a larger corpus, explore Kneser-Ney-inspired smoothing of retrieval scores, or investigate fine-tuning Qwen directly to combine the strengths of both paradigms.